GMoney E-Wallet Monolith

Test Cases and Test Scripts

MCO1 Deliverable 3 — GCash FinTech Engineer Specialist Course

Richard Limtin · October 29, 2026

688	688	0	0
tests executed	passed	failed	methods with < 3 cases

Every method in every class of `src/main/java` is tested with at least three documented, unique test cases. This document lists each case with its scenario, expected result and the result of the latest run, and names the JUnit test script that executes it.

1. Running the test scripts

Every test case below is an executable JUnit 5 test in `src/test/java`. Tests use an in-memory H2 database (`src/test/resources/application.properties`), so MySQL does not need to be running and no data in `gmoney_db` is touched.

Goal	Command
Run every test	`./mvnw test`
Run one test class	`./mvnw test -Dtest=TransferServiceTest`
Run one test method	`./mvnw test -Dtest=TransferServiceTest#transferMoney_zeroAmount`
Run from IntelliJ IDEA	Right-click src/test/java > Run 'All Tests'
Results	`target/surefire-reports (one XML and TXT report per test class)`

Latest run: 688 tests executed, 688 passed, 0 failed, 0 skipped (Maven Surefire, 2026-10-29).

2. Test design

Tests are written at three levels, following the Week 6 topics (JUnit and Mockito, MockMvc slice tests and integration tests). Each level checks something the others cannot.

Level	Tools	What it proves	Cases
Unit	JUnit 5, Mockito (@Mock, @InjectMocks)	Business rules in services, security helpers, the forex client and entities, with the database and other beans replaced by mocks.	213
Controller slice	@WebMvcTest, MockMvc, @MockitoBean	HTTP routes, status codes, JSON bodies, bean validation and the error handler, with the service mocked.	98
Integration	@SpringBootTest, MockMvc, H2	The whole application together: real security filter chain and JWTs, transactions, rollback, database constraints and concurrent transfers.	70

Test case IDs. Every case has a unique ID and a three-part name used as its JUnit `@DisplayName`: `TC-TRFSVC-08 | TransferService.transferMoney | amount 0 -> InvalidTransactionException` = case ID, method under test, scenario -> expected result. The prefix names the test class (for example TRFSVC = TransferServiceTest). No ID or scenario is repeated.

Three unique cases per method. Cases for the same method differ in input and expected outcome, typically a success path, a rejected input and an edge or security case.

Special cases. Getters and setters are checked by a parameterized test with three sample values each (Section 5). Private helpers cannot be called directly; each is covered by three named cases that execute it (Section 6). Constructors are listed in Section 7.

3. Coverage summary

275 methods across 45 classes. The fourth column is the smallest number of passing test cases for any single method in that class; every class reaches at least three.

Class	Package	Methods	Fewest cases per method	Status
GMoneyApplication	walletapi	1	3	Meets 3+
FrankfurterClient	client	2	3	Meets 3+
DataSeeder	config	1	3	Meets 3+
ForexRateSyncScheduler	config	3	3	Meets 3+
AccountController	controller	2	3	Meets 3+
AuthController	controller	4	3	Meets 3+
CustomerController	controller	3	3	Meets 3+
ForexController	controller	4	3	Meets 3+
TransferController	controller	2	3	Meets 3+
AccountDTO	dto	16	3	Meets 3+
ApiResponse	dto	8	3	Meets 3+
BalanceUpdateDTO	dto	2	3	Meets 3+
CustomerDTO	dto	10	3	Meets 3+
ExchangeRateDTO	dto	10	3	Meets 3+
ForexQuoteDTO	dto	10	3	Meets 3+
ForexSyncResultDTO	dto	8	3	Meets 3+
LoginDTO	dto	4	3	Meets 3+
RegisterDTO	dto	10	3	Meets 3+
TransferRecordDTO	dto	20	3	Meets 3+
TransferRequestDTO	dto	6	3	Meets 3+
Account	entity	16	3	Meets 3+
Customer	entity	10	3	Meets 3+
ExchangeRate	entity	10	3	Meets 3+
SupportedCurrency	entity	3	3	Meets 3+
TransferRecord	entity	19	3	Meets 3+
User	entity	15	3	Meets 3+
ErrorResponseDTO	exception	8	3	Meets 3+
GlobalExceptionHandler	exception	11	3	Meets 3+
AccountRepository	repository	4	3	Meets 3+
CustomerRepository	repository	1	3	Meets 3+
ExchangeRateRepository	repository	2	3	Meets 3+
TransferRecordRepository	repository	1	3	Meets 3+
UserRepository	repository	1	3	Meets 3+
AccountSecurity	security	1	5	Meets 3+
CustomAccessDeniedHandler	security	1	3	Meets 3+
CustomAuthenticationEntryPoint	security	1	3	Meets 3+
CustomUserDetailsService	security	1	3	Meets 3+
JwtAuthenticationFilter	security	1	4	Meets 3+
JwtUtil	security	5	3	Meets 3+
SecurityConfig	security	4	3	Meets 3+
AccountService	service	9	3	Meets 3+
CustomerService	service	7	3	Meets 3+
ExchangeRateService	service	10	3	Meets 3+
ForexRateSyncService	service	2	3	Meets 3+

Class	Package	Methods	Fewest cases per method	Status
TransferService	service	6	3	**Meets 3+**

4. Test case catalogue

Grouped by class and method. *Script* is the JUnit class and test method to run.

GMoneyApplication

GMoneyApplication.main (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-APP-01	Standard call	Spring context loads	**Pass**	`GMoneyApplicationTests #contextLoads`
TC-APP-02	Standard call	transfer and forex beans are wired; application name is gmoney	**Pass**	`GMoneyApplicationTests #coreBeansPresent`
TC-APP-03	Standard call	main() boots a separate app instance without errors	**Pass**	`GMoneyApplicationTests #mainRuns`

FrankfurterClient

FrankfurterClient.fetchLatestRate (5 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-FXCLI-01	USD->PHP 63.944	rate and ECB date parsed	**Pass**	`FrankfurterClientTest #fetch_parsesRateAndDate`
TC-FXCLI-02	API answers 404	RestClientException	**Pass**	`FrankfurterClientTest #fetch_httpError`
TC-FXCLI-03	API down (503)	RestClientException	**Pass**	`FrankfurterClientTest #fetch_serverError`
TC-FXCLI-04	response without the quote currency	RestClientException	**Pass**	`FrankfurterClientTest #fetch_missingQuote`
TC-FXCLI-05	zero rate	RestClientException	**Pass**	`FrankfurterClientTest #fetch_zeroRate`

DataSeeder

DataSeeder.run (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-SEED-01	empty database	admin (ROLE_ADMIN, hashed password) and 4 rates created	**Pass**	`DataSeederTest #run_emptyDatabase`
TC-SEED-02	admin and rates already exist	nothing saved (idempotent restart)	**Pass**	`DataSeederTest #run_alreadySeeded`
TC-SEED-03	only USD->PHP missing	only that rate is added, admin kept	**Pass**	`DataSeederTest #run_partialRates`

ForexRateSyncScheduler

ForexRateSyncScheduler.syncOnStartup (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-FXSCH-01	application ready	one sync is run	**Pass**	`ForexRateSyncSchedulerTest #startup_runsSync`
TC-FXSCH-02	provider down	startup continues, no exception	**Pass**	`ForexRateSyncSchedulerTest #startup_providerDown`
TC-FXSCH-03	unexpected bug	not swallowed (propagates)	**Pass**	`ForexRateSyncSchedulerTest #startup_unexpectedError`

ForexRateSyncScheduler.syncOnSchedule (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-FXSCH-04	cron fires	one sync is run	Pass	`ForexRateSyncSchedulerTest #schedule_runsSync`
TC-FXSCH-05	provider down	logged, scheduler keeps running	Pass	`ForexRateSyncSchedulerTest #schedule_providerDown`
TC-FXSCH-06	fires twice	two independent syncs	Pass	`ForexRateSyncSchedulerTest #schedule_twice`

AccountController

AccountController.createAccount (7 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-ACCCTL-04	USD forex wallet	201 with currency USD	Pass	`AccountControllerTest #create_usdWallet`
TC-ACCCTL-05	negative balance	400 'Balance cannot be negative'	Pass	`AccountControllerTest #create_negativeBalance`
TC-ACCCTL-06	currency 'US' (not 3 letters)	400	Pass	`AccountControllerTest #create_badCurrencyFormat`
TC-ACCCTL-07	unsupported currency JPY	400 'Invalid Transaction'	Pass	`AccountControllerTest #create_unsupportedCurrency`
TC-ACCCTL-23	second USD wallet for the customer	409 Conflict	Pass	`AccountControllerTest #create_duplicateCurrency`
TC-ACCINT-04	second USD wallet for one customer	201 then 409, one row kept	Pass	`AccountIntegrationTest #testCreateAccount_SecondWalletSameCurrency_Returns409`
TC-ACCINT-05	one customer opens PHP, USD and EUR wallets	all 201	Pass	`AccountIntegrationTest #testCreateAccount_DifferentCurrencies_AllAllowed`

AccountController.getMyAccounts (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-ACCCTL-11	alice	her PHP and USD wallets	Pass	`AccountControllerTest #mine_twoWallets`
TC-ACCCTL-12	user with no wallets	empty list	Pass	`AccountControllerTest #mine_none`
TC-ACCCTL-13	Standard call	username taken from the token, not the URL	Pass	`AccountControllerTest #mine_usesToken`

AuthController

AuthController.register (8 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-AUTHCTL-01	new username	201, password hashed, ROLE_CUSTOMER with profile	Pass	`AuthControllerTest #register_ok`
TC-AUTHCTL-02	username taken	409 'Username already exists'	Pass	`AuthControllerTest #register_duplicate`
TC-AUTHCTL-03	invalid e-mail	400 'Email must be a valid email address'	Pass	`AuthControllerTest #register_badEmail`
TC-AUTHCTL-10	phone +639171234567	201, phone saved on the profile	Pass	`AuthControllerTest #register_internationalPhone`
TC-AUTHCTL-11	no phone given	201 (phone is optional)	Pass	`AuthControllerTest #register_withoutPhone`
TC-AUTHCTL-12	phone '12345' (too short)	400 with phone message, nothing saved	Pass	`AuthControllerTest #register_shortPhone`
TC-AUTHCTL-13	phone '0917-123-4567' (dashes)	400	Pass	`AuthControllerTest #register_dashedPhone`
TC-AUTHCTL-14	Swagger placeholder phone 'string'	400	Pass	`AuthControllerTest #register_placeholderPhone`

AuthController.registerAdmin (4 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-AUTHCTL-04	admin creates admin	201, ROLE_ADMIN saved	Pass	`AuthControllerTest #registerAdmin_ok`
TC-AUTHCTL-05	username taken	409	Pass	`AuthControllerTest #registerAdmin_duplicate`
TC-AUTHCTL-06	missing password	400 'Password is required'	Pass	`AuthControllerTest #registerAdmin_missingPassword`
TC-AUTHCTL-15	phone '0817123456' (wrong prefix)	400, nothing saved	Pass	`AuthControllerTest #registerAdmin_badPhone`

AuthController.login (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-AUTHCTL-07	correct credentials	200 with token in data	Pass	`AuthControllerTest #login_ok`
TC-AUTHCTL-08	wrong password	401 'Invalid username or password', no token	Pass	`AuthControllerTest #login_badCredentials`
TC-AUTHCTL-09	blank username	400 'Username is required'	Pass	`AuthControllerTest #login_blankUsername`

CustomerController

CustomerController.getCurrentCustomer (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-CUSCTL-01	logged-in alice	200 with her profile	Pass	`CustomerControllerTest #me_ok`
TC-CUSCTL-02	user without profile	404	Pass	`CustomerControllerTest #me_noProfile`
TC-CUSCTL-03	Standard call	username comes from the token	Pass	`CustomerControllerTest #me_usesToken`

CustomerController.createCustomer (5 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-CUSCTL-04	valid body	201	Pass	`CustomerControllerTest #create_ok`
TC-CUSCTL-05	blank name	400 'Customer name is required'	Pass	`CustomerControllerTest #create_blankName`
TC-CUSCTL-06	non-admin	403	Pass	`CustomerControllerTest #create_forbidden`
TC-CUSCTL-19	phone '+639171234567'	201	Pass	`CustomerControllerTest #createWithInternationalPhone`
TC-CUSCTL-20	phone 'abc'	400 with phone message, service not called	Pass	`CustomerControllerTest #createWithInvalidPhone`

CustomerController.getAllCustomers (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-CUSCTL-07	populated list	200 with profiles	Pass	`CustomerControllerTest #listCustomers`
TC-CUSCTL-08	empty list	empty JSON array	Pass	`CustomerControllerTest #listEmpty`
TC-CUSCTL-09	forbidden service	403	Pass	`CustomerControllerTest #listForbidden`

ForexController

ForexController.getSupportedCurrencies (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-FXCTL-01	Standard call	-> 200 with PHP 'Philippine Peso'	Pass	`ForexControllerTest #currencies_php`
TC-FXCTL-02	Standard call	-> USD present	Pass	`ForexControllerTest #currencies_usd`
TC-FXCTL-03	Standard call	-> unsupported JPY absent, message set	Pass	`ForexControllerTest #currencies_noJpy`

ForexController.getAllRates (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-FXCTL-04	two rates	200 with both	Pass	`ForexControllerTest #rates_list`
TC-FXCTL-05	none	200 with empty list	Pass	`ForexControllerTest #rates_empty`
TC-FXCTL-06	Standard call	message 'Exchange rates retrieved'	Pass	`ForexControllerTest #rates_message`

ForexController.createRate (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-FXCTL-10	valid body	201 'Exchange rate created'	Pass	`ForexControllerTest #createRate_ok`
TC-FXCTL-11	negative rate	400 'Rate must be greater than zero'	Pass	`ForexControllerTest #createRate_negative`
TC-FXCTL-12	duplicate pair	400 from InvalidTransaction Exception	Pass	`ForexControllerTest #createRate_duplicate`

ForexController.syncRates (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-FXCTL-22	admin	200 with live rates and provider	Pass	`ForexControllerTest #sync_admin`
TC-FXCTL-23	provider unreachable	503	Pass	`ForexControllerTest #sync_providerDown`
TC-FXCTL-24	GBP not refreshed	200 listing GBP in failedCurrencies	Pass	`ForexControllerTest #sync_partialFailure`

TransferController

TransferController.transferMoney (7 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-TRFCTL-01	valid body + JWT user	200 'Transfer successful', username from token	Pass	`TransferControllerTest #transfer_success`
TC-TRFCTL-02	missing fromId	400 'Source account ID is required'	Pass	`TransferControllerTest #transfer_missingFromId`
TC-TRFCTL-03	amount 0	400 'Transfer amount must be greater than zero'	Pass	`TransferControllerTest #transfer_zeroAmount`
TC-TRFCTL-04	missing toId	400 'Destination account ID is required'	Pass	`TransferControllerTest #transfer_missingToId`
TC-TRFCTL-05	destination 999 missing	404 'Destination account not found'	Pass	`TransferControllerTest #transfer_destinationMissing`
TC-TRFCTL-06	sender is not the owner	403	Pass	`TransferControllerTest #transfer_notOwner`
TC-TRFCTL-07	nobody logged in	401, service never called	Pass	`TransferControllerTest #transfer_noUser`

Account

Account.onCreate (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-ENT-01	new account	createdAt and updateAt set	Pass	`EntityLifecycleTest #account_onCreate_timestamps`
TC-ENT-02	no currency	defaults to PHP	Pass	`EntityLifecycleTest #account_onCreate_defaultCurrency`
TC-ENT-03	USD given	kept as USD	Pass	`EntityLifecycleTest #account_onCreate_keepsCurrency`

Account.onUpdate (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-ENT-04	update	updateAt refreshed	Pass	`EntityLifecycleTest #account_onUpdate_refreshes`
TC-ENT-05	update	createdAt unchanged	Pass	`EntityLifecycleTest #account_onUpdate_keepsCreated`
TC-ENT-06	Standard call	currency and balance untouched	Pass	`EntityLifecycleTest #account_onUpdate_keepsData`

Account.getCreatedAt (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-ENT-01	new account	createdAt and updateAt set	Pass	`EntityLifecycleTest #account_onCreate_timestamps`
TC-ENT-05	update	createdAt unchanged	Pass	`EntityLifecycleTest #account_onUpdate_keepsCreated`
TC-ENT-19	never persisted	null until onCreate runs	Pass	`EntityLifecycleTest #account_getCreatedAt_beforePersist`

Account.getUpdateAt (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-ENT-01	new account	createdAt and updateAt set	Pass	`EntityLifecycleTest #account_onCreate_timestamps`
TC-ENT-04	update	updateAt refreshed	Pass	`EntityLifecycleTest #account_onUpdate_refreshes`
TC-ENT-20	never persisted	null until onCreate runs	Pass	`EntityLifecycleTest #account_getUpdateAt_beforePersist`

ExchangeRate

ExchangeRate.touch (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-ENT-10	insert/update	updatedAt set	Pass	`EntityLifecycleTest #rate_touch_sets`
TC-ENT-11	second touch	later timestamp	Pass	`EntityLifecycleTest #rate_touch_refreshes`
TC-ENT-12	Standard call	pair and rate untouched	Pass	`EntityLifecycleTest #rate_touch_keepsData`

ExchangeRate.getUpdatedAt (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-ENT-10	insert/update	updatedAt set	Pass	`EntityLifecycleTest #rate_touch_sets`
TC-ENT-11	second touch	later timestamp	Pass	`EntityLifecycleTest #rate_touch_refreshes`
TC-REPO-18	Standard call	updatedAt stamped on save	Pass	`RepositoryIntegrationTest #rate_findTimestamp`

SupportedCurrency

SupportedCurrency.getDisplayName (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-CUR-07	PHP	'Philippine Peso'	**Pass**	`SupportedCurrencyTest #displayName_php`
TC-CUR-08	USD	'US Dollar'	**Pass**	`SupportedCurrencyTest #displayName_usd`
TC-CUR-09	Standard call	every constant has a non-blank name; HOME is PHP	**Pass**	`SupportedCurrencyTest #displayName_all`

SupportedCurrency.isSupported (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-CUR-01	'USD'	true	**Pass**	`SupportedCurrencyTest #isSupported_upper`
TC-CUR-02	' php ' (lower case, spaces)	true	**Pass**	`SupportedCurrencyTest #isSupported_lowerWithSpaces`
TC-CUR-03	'JPY', blank and null	false	**Pass**	`SupportedCurrencyTest #isSupported_unsupported`

SupportedCurrency.fromCode (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-CUR-04	'eur'	EUR	**Pass**	`SupportedCurrencyTest #fromCode_valid`
TC-CUR-05	'BTC'	InvalidTransactionException listing supported codes	**Pass**	`SupportedCurrencyTest #fromCode_invalid`
TC-CUR-06	null	InvalidTransactionException	**Pass**	`SupportedCurrencyTest #fromCode_null`

TransferRecord

TransferRecord.onCreate (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-ENT-07	insert	transferDate set	**Pass**	`EntityLifecycleTest #transfer_onCreate_sets`
TC-ENT-08	Standard call	transferDate is 'now' (within 5 seconds)	**Pass**	`EntityLifecycleTest #transfer_onCreate_now`
TC-ENT-09	Standard call	amount and ids untouched	**Pass**	`EntityLifecycleTest #transfer_onCreate_keepsData`

User

User.getAuthorities (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-ENT-13	default role	ROLE_CUSTOMER	**Pass**	`EntityLifecycleTest #user_defaultRole`
TC-ENT-14	ROLE_ADMIN	ROLE_ADMIN	**Pass**	`EntityLifecycleTest #user_adminRole`
TC-ENT-15	blank or null role	falls back to ROLE_CUSTOMER	**Pass**	`EntityLifecycleTest #user_blankRole`

User.isAccountNonExpired (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-ENT-16	new user	all true	**Pass**	`EntityLifecycleTest #user_flags_new`
TC-ENT-17	admin	all true	**Pass**	`EntityLifecycleTest #user_flags_admin`
TC-ENT-18	user with profile	all true	**Pass**	`EntityLifecycleTest #user_flags_withProfile`

User.isAccountNonLocked (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-ENT-16	new user	all true	Pass	`EntityLifecycleTest #user_flags_new`
TC-ENT-17	admin	all true	Pass	`EntityLifecycleTest #user_flags_admin`
TC-ENT-18	user with profile	all true	Pass	`EntityLifecycleTest #user_flags_withProfile`

User.isCredentialsNonExpired (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-ENT-16	new user	all true	Pass	`EntityLifecycleTest #user_flags_new`
TC-ENT-17	admin	all true	Pass	`EntityLifecycleTest #user_flags_admin`
TC-ENT-18	user with profile	all true	Pass	`EntityLifecycleTest #user_flags_withProfile`

User.isEnabled (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-ENT-16	new user	all true	Pass	`EntityLifecycleTest #user_flags_new`
TC-ENT-17	admin	all true	Pass	`EntityLifecycleTest #user_flags_admin`
TC-ENT-18	user with profile	all true	Pass	`EntityLifecycleTest #user_flags_withProfile`

GlobalExceptionHandler

GlobalExceptionHandler.handleValidationErrors (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-GEH-01	amount field error	400 with DTO message	Pass	`GlobalExceptionHandlerTest #validation_amount`
TC-GEH-02	Standard call	errorType is 'Validation Failed'	Pass	`GlobalExceptionHandlerTest #validation_errorType`
TC-GEH-03	no field error (object-level)	'Invalid request'	Pass	`GlobalExceptionHandlerTest #validation_noFieldError`

GlobalExceptionHandler.handleNotFound (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-GEH-04	'Destination account not found'	404 same message	Pass	`GlobalExceptionHandlerTest #notFound_destination`
TC-GEH-05	'Insufficient funds'	404 (course contract)	Pass	`GlobalExceptionHandlerTest #notFound_insufficientFunds`
TC-GEH-06	Standard call	errorType is 'Not Found'	Pass	`GlobalExceptionHandlerTest #notFound_errorType`

GlobalExceptionHandler.handleInvalidTransaction (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-GEH-07	same-account transfer	400 same message	Pass	`GlobalExceptionHandlerTest #invalid_sameAccount`
TC-GEH-08	Standard call	errorType is 'Invalid Transaction'	Pass	`GlobalExceptionHandlerTest #invalid_errorType`
TC-GEH-09	Standard call	unsupported currency message passed through	Pass	`GlobalExceptionHandlerTest #invalid_currency`

GlobalExceptionHandler.handleDuplicateWallet (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-GEH-25	second USD wallet	409	Pass	`GlobalExceptionHandlerTest #duplicateWallet_status`
TC-GEH-26	Standard call	errorType is 'Conflict'	Pass	`GlobalExceptionHandlerTest #duplicateWallet_errorType`
TC-GEH-27	Standard call	exception message passed to the client	Pass	`GlobalExceptionHandlerTest #duplicateWallet_message`

GlobalExceptionHandler.handleDataIntegrityViolation (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-GEH-28	unique key rejects a write	409	Pass	`GlobalExceptionHandlerTest #dataIntegrity_status`
TC-GEH-29	Standard call	errorType is 'Conflict'	Pass	`GlobalExceptionHandlerTest #dataIntegrity_errorType`
TC-GEH-30	Standard call	SQL details are not leaked to the client	Pass	`GlobalExceptionHandlerTest #dataIntegrity_safeMessage`

GlobalExceptionHandler.handleProviderUnavailable (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-GEH-22	live-rate provider down	503	Pass	`GlobalExceptionHandlerTest #providerUnavailable_status`
TC-GEH-23	Standard call	errorType is 'Service Unavailable'	Pass	`GlobalExceptionHandlerTest #providerUnavailable_errorType`
TC-GEH-24	Standard call	exception message passed to the client	Pass	`GlobalExceptionHandlerTest #providerUnavailable_message`

GlobalExceptionHandler.handleBadCredentials (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-GEH-10	bad password	401	Pass	`GlobalExceptionHandlerTest #badCredentials_status`
TC-GEH-11	Standard call	message never reveals which field was wrong	Pass	`GlobalExceptionHandlerTest #badCredentials_message`
TC-GEH-12	Standard call	errorType is 'Unauthorized'	Pass	`GlobalExceptionHandlerTest #badCredentials_errorType`

GlobalExceptionHandler.handleAccessDenied (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-GEH-13	not the owner	403	Pass	`GlobalExceptionHandlerTest #accessDenied_status`
TC-GEH-14	Standard call	generic permission message	Pass	`GlobalExceptionHandlerTest #accessDenied_message`
TC-GEH-15	Standard call	errorType is 'Forbidden'	Pass	`GlobalExceptionHandlerTest #accessDenied_errorType`

GlobalExceptionHandler.handleBadRequest (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-GEH-16	malformed JSON body	400	Pass	`GlobalExceptionHandlerTest #badRequest_json`
TC-GEH-17	non-numeric id in URL	400	Pass	`GlobalExceptionHandlerTest #badRequest_typeMismatch`
TC-GEH-18	Standard call	fixed safe message	Pass	`GlobalExceptionHandlerTest #badRequest_message`

GlobalExceptionHandler.handleConcurrentUpdate (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-GEH-40	stale update	409	Pass	`GlobalExceptionHandlerTest #concurrentUpdateStatus`
TC-GEH-41	conflict body	consistent error type	Pass	`GlobalExceptionHandlerTest #concurrentUpdateShape`
TC-GEH-42	database details	not exposed	Pass	`GlobalExceptionHandlerTest #concurrentUpdateSafeMessage`

GlobalExceptionHandler.handleEverythingElse (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-GEH-19	Spring web error (404 status exception)	keeps 404	Pass	`GlobalExceptionHandlerTest #everythingElse_webError`
TC-GEH-20	authentication problem	401	Pass	`GlobalExceptionHandlerTest #everythingElse_auth`
TC-GEH-21	unexpected bug	500 with safe message	Pass	`GlobalExceptionHandlerTest #everythingElse_bug`

AccountRepository

AccountRepository.findByStatus (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-REPO-01	one FROZEN-TEST wallet	found	Pass	`RepositoryIntegrationTest #findByStatus_match`
TC-REPO-02	unused status	empty	Pass	`RepositoryIntegrationTest #findByStatus_none`
TC-REPO-03	mixed statuses	only exact matches	Pass	`RepositoryIntegrationTest #findByStatus_exact`

AccountRepository.findByCustomer_User_Username (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-REPO-04	user with PHP + USD wallets	2	Pass	`RepositoryIntegrationTest #findByUsername_two`
TC-REPO-05	Standard call	other user's wallets excluded	Pass	`RepositoryIntegrationTest #findByUsername_isolated`
TC-REPO-06	unknown user	empty	Pass	`RepositoryIntegrationTest #findByUsername_unknown`

AccountRepository.existsByCustomer_IdAndCurrency (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-REPO-22	customer holds a USD wallet	true for USD	Pass	`RepositoryIntegrationTest #existsByCustomerAndCurrency_match`
TC-REPO-23	customer holds USD only	false for EUR	Pass	`RepositoryIntegrationTest #existsByCustomerAndCurrency_otherCurrency`
TC-REPO-26	another customer's USD wallet	false	Pass	`RepositoryIntegrationTest #existsByCustomerAndCurrency_otherCustomer`

AccountRepository.existsByCustomer_Id (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-REPO-27	owner with wallet	true	Pass	`RepositoryIntegrationTest #customerWithWalletExists`
TC-REPO-28	customer without wallets	false	Pass	`RepositoryIntegrationTest #customerWithoutWallets`
TC-REPO-29	another customer's wallet	false	Pass	`RepositoryIntegrationTest #otherCustomersWalletDoesNotMatch`

CustomerRepository

CustomerRepository.findByFullNameContaining (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-REPO-07	'Zyx' inside 'Zyxwv Reyes'	found	Pass	`RepositoryIntegrationTest #nameContaining_match`
TC-REPO-08	no match	empty	Pass	`RepositoryIntegrationTest #nameContaining_none`
TC-REPO-09	keyword in two names	both	Pass	`RepositoryIntegrationTest #nameContaining_many`

ExchangeRateRepository

ExchangeRateRepository.findByBaseCurrencyAndQuoteCurrency (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-REPO-16	stored SGD->USD	found	Pass	`RepositoryIntegrationTest #rate_find`
TC-REPO-17	reverse direction is a different row	empty	Pass	`RepositoryIntegrationTest #rate_findReverse`
TC-REPO-18	Standard call	updatedAt stamped on save	Pass	`RepositoryIntegrationTest #rate_findTimestamp`

ExchangeRateRepository.existsByBaseCurrencyAndQuoteCurrency (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-REPO-19	stored pair	true	Pass	`RepositoryIntegrationTest #rate_existsTrue`
TC-REPO-20	missing pair	false	Pass	`RepositoryIntegrationTest #rate_existsFalse`
TC-REPO-21	pair deleted	false	Pass	`RepositoryIntegrationTest #rate_existsAfterDelete`

TransferRecordRepository

TransferRecordRepository.findBySourceAccountIdOrDestinationAccountId (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-REPO-13	sent + received	both	Pass	`RepositoryIntegrationTest #ledger_bothDirections`
TC-REPO-14	Standard call	unrelated rows excluded	Pass	`RepositoryIntegrationTest #ledger_excludesOthers`
TC-REPO-15	no transfers	empty; transferDate auto-set	Pass	`RepositoryIntegrationTest #ledger_emptyAndTimestamp`

UserRepository

UserRepository.findByUsername (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-REPO-10	existing	present	Pass	`RepositoryIntegrationTest #findUser_present`
TC-REPO-11	missing	empty	Pass	`RepositoryIntegrationTest #findUser_missing`
TC-REPO-12	Standard call	profile is linked to the found user	Pass	`RepositoryIntegrationTest #findUser_linkedProfile`

AccountSecurity

AccountSecurity.isOwnCustomer (5 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-ACCSEC-01	alice uses her own customerId 5	true	Pass	`AccountSecurityTest #ownCustomer_true`
TC-ACCSEC-02	alice uses Bob's customerId 6	false	Pass	`AccountSecurityTest #otherCustomer_false`
TC-ACCSEC-03	null customerId	false without DB lookup	Pass	`AccountSecurityTest #nullCustomer_false`
TC-ACCSEC-04	unknown user	false	Pass	`AccountSecurityTest #unknownUser_false`
TC-ACCSEC-05	user without customer profile	false	Pass	`AccountSecurityTest #userWithoutProfile_false`

CustomAccessDeniedHandler

CustomAccessDeniedHandler.handle (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-ADH-01	denied request	status 403	Pass	`CustomErrorWritersTest #handle_status`
TC-ADH-02	Standard call	content type is application/json	Pass	`CustomErrorWritersTest #handle_contentType`
TC-ADH-03	Standard call	body carries 'Forbidden' and the permission message	Pass	`CustomErrorWritersTest #handle_body`

CustomAuthenticationEntryPoint

CustomAuthenticationEntryPoint.commence (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-AEP-01	unauthenticated request	status 401	Pass	`CustomErrorWritersTest #commence_status`
TC-AEP-02	Standard call	content type is application/json	Pass	`CustomErrorWritersTest #commence_contentType`
TC-AEP-03	Standard call	body carries the standard 401 message	Pass	`CustomErrorWritersTest #commence_body`

CustomUserDetailsService

CustomUserDetailsService.loadUserByUsername (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-UDS-01	existing user	same user returned	Pass	`CustomUserDetailsServiceTest #load_existing`
TC-UDS-02	unknown user	UsernameNotFoundException	Pass	`CustomUserDetailsServiceTest #load_missing`
TC-UDS-03	admin	authorities contain ROLE_ADMIN	Pass	`CustomUserDetailsServiceTest #load_adminAuthorities`

JwtAuthenticationFilter

JwtAuthenticationFilter.doFilterInternal (4 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-JWTF-01	no Authorization header	chain continues, nobody authenticated	Pass	`JwtAuthenticationFilterTest #noHeader`
TC-JWTF-02	valid Bearer token	user authenticated with roles	Pass	`JwtAuthenticationFilterTest #validToken`
TC-JWTF-03	malformed token	context cleared, chain continues	Pass	`JwtAuthenticationFilterTest #invalidToken`
TC-JWTF-04	'Basic' scheme instead of Bearer	ignored	Pass	`JwtAuthenticationFilterTest #nonBearerHeader`

JwtUtil

JwtUtil.generateToken (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-JWT-01	username alice	three-part signed JWT	Pass	`JwtUtilTest #generateToken_format`
TC-JWT-02	different users	different tokens	Pass	`JwtUtilTest #generateToken_unique`
TC-JWT-03	Standard call	token subject round-trips to the same username	Pass	`JwtUtilTest #generateToken_roundTrip`

JwtUtil.extractUsername (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-JWT-04	valid token	subject	Pass	`JwtUtilTest #extractUsername_valid`
TC-JWT-05	tampered signature	JwtException	Pass	`JwtUtilTest #extractUsername_tampered`
TC-JWT-06	garbage string	JwtException	Pass	`JwtUtilTest #extractUsername_garbage`

JwtUtil.isTokenValid (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-JWT-07	token for alice checked against alice	true	Pass	`JwtUtilTest #isTokenValid_matchingUser`
TC-JWT-08	token for alice checked against bob	false	Pass	`JwtUtilTest #isTokenValid_otherUser`
TC-JWT-09	case differs (Alice vs alice)	false	Pass	`JwtUtilTest #isTokenValid_caseSensitive`

SecurityConfig

SecurityConfig.securityFilterChain (7 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-SEC-01	/actuator/health without token	200 (public)	Pass	`SecurityConfigTest #chain_healthIsPublic`
TC-SEC-02	/api/v1/forex/rates without token	401 (protected)	Pass	`SecurityConfigTest #chain_protectedNeedsToken`
TC-SEC-03	customer token on admin-only POST /forex/rates	403	Pass	`SecurityConfigTest #chain_customerBlockedFromAdminEndpoint`
TC-SEC-04	admin token on POST /forex/rates	201	Pass	`SecurityConfigTest #chain_adminAllowed`
TC-SEC-14	/v3/api-docs without token	200 with JWT bearer scheme	Pass	`SecurityConfigTest #chain_apiDocsArePublic`
TC-SEC-15	/swagger-ui/index.html without token	200 (public)	Pass	`SecurityConfigTest #chain_swaggerUiIsPublic`
TC-SEC-16	customer token on admin-only POST /forex/rates/sync	403	Pass	`SecurityConfigTest #chain_customerBlockedFromRateSync`

SecurityConfig.authenticationProvider (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-SEC-11	Standard call	supports username/password tokens	Pass	`SecurityConfigTest #provider_supports`
TC-SEC-12	valid admin credentials	ROLE_ADMIN authority	Pass	`SecurityConfigTest #provider_roles`
TC-SEC-13	wrong password	BadCredentialsException	Pass	`SecurityConfigTest #provider_wrongPassword`

SecurityConfig.authenticationManager (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-SEC-08	correct credentials	authenticated	Pass	`SecurityConfigTest #manager_valid`
TC-SEC-09	wrong password	BadCredentialsException	Pass	`SecurityConfigTest #manager_wrongPassword`
TC-SEC-10	unknown user	BadCredentialsException (no user enumeration)	Pass	`SecurityConfigTest #manager_unknownUser`

SecurityConfig.passwordEncoder (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-SEC-05	Standard call	hash is not the plain text and matches it	Pass	`SecurityConfigTest #encoder_hashesAndMatches`
TC-SEC-06	Standard call	wrong password does not match	Pass	`SecurityConfigTest #encoder_wrongPassword`
TC-SEC-07	same password hashed twice	different salts	Pass	`SecurityConfigTest #encoder_salted`

AccountService

AccountService.createAccount (8 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-ACCSVC-04	no currency given	PHP wallet linked to customer	Pass	`AccountServiceTest #createAccount_defaultsToPhp`
TC-ACCSVC-05	currency 'usd'	forex wallet stored as USD	Pass	`AccountServiceTest #createAccount_forexWallet_normalizesCurrency`
TC-ACCSVC-06	unsupported currency JPY	InvalidTransactionExceptio n, nothing saved	Pass	`AccountServiceTest #createAccount_unsupportedCurrency_throws`
TC-ACCSVC-07	unknown customerId	IllegalArgumentException 'Customer not Found'	Pass	`AccountServiceTest #createAccount_unknownCustomer_throws`
TC-ACCSVC-26	customer already has a USD wallet	DuplicateWalletException, nothing saved	Pass	`AccountServiceTest #createAccount_duplicateCurrency_throws`
TC-ACCSVC-27	no currency given + existing PHP wallet	DuplicateWalletException (default PHP is checked)	Pass	`AccountServiceTest #createAccount_duplicateDefaultPhp_throws`
TC-ACCSVC-28	currency 'eur'	duplicate check uses normalized 'EUR', new currency allowed	Pass	`AccountServiceTest #createAccount_checksNormalizedCurrency`
TC-ACCSVC-29	no customerId (admin, unowned wallet)	no duplicate check, saved	Pass	`AccountServiceTest #createAccount_noCustomer_skipsDuplicateCheck`

AccountService.getAllAccounts (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-ACCSVC-08	two accounts in DB	two DTOs	Pass	`AccountServiceTest #getAllAccounts_returnsAll`
TC-ACCSVC-09	empty DB	empty list	Pass	`AccountServiceTest #getAllAccounts_empty`
TC-ACCSVC-10	account without customer	DTO with null owner fields	Pass	`AccountServiceTest #getAllAccounts_unlinkedAccount`

AccountService.getMyAccounts (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-ACCSVC-11	alice owns PHP + USD wallets	both returned	Pass	`AccountServiceTest #getMyAccounts_returnsOwnedWallets`
TC-ACCSVC-12	user with no wallets	empty list	Pass	`AccountServiceTest #getMyAccounts_none`
TC-ACCSVC-13	Standard call	queries the repository with the given username only	Pass	`AccountServiceTest #getMyAccounts_usesUsername`

AccountService.getAccountById (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-ACCSVC-01	unknown id 99	IllegalArgumentException 'Account not Found'	Pass	`AccountServiceTest #testGetAccountById_NotFound_ThrowsException`
TC-ACCSVC-02	existing id 1	DTO with accountType	Pass	`AccountServiceTest #testGetAccountById_Found_ReturnsDTO`
TC-ACCSVC-03	owned USD wallet	DTO carries owner username and currency	Pass	`AccountServiceTest #getAccountById_ownedForexWallet_mapsOwnerAndCurrency`

AccountService.getAccountByStatus (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-ACCSVC-14	ACTIVE	matching accounts	Pass	`AccountServiceTest #getAccountByStatus_active`
TC-ACCSVC-15	FROZEN with no matches	empty list	Pass	`AccountServiceTest #getAccountByStatus_noMatch`
TC-ACCSVC-16	Standard call	status value passed to repository unchanged	Pass	`AccountServiceTest #getAccountByStatus_passesStatus`

AccountService.closeAccount (6 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-ACCSVC-17	existing id	true and deleted	Pass	`AccountServiceTest #closeAccount_existing`
TC-ACCSVC-18	unknown id	false, nothing deleted	Pass	`AccountServiceTest #closeAccount_missing`
TC-ACCSVC-19	Standard call	deletes only the requested id	Pass	`AccountServiceTest #closeAccount_onlyRequestedId`
TC-ACCSVC-30	funded wallet	conflict, no deletion	Pass	`AccountServiceTest #closeFundedWalletRejected`
TC-ACCSVC-31	zero balance with history	conflict, history preserved	Pass	`AccountServiceTest #closeHistoricalWalletRejected`
TC-CUSINT-04	funded or historical wallet	409 without destroying money or history	Pass	`CustomerLifecycleIntegrationTest #protectedWalletDeletion`

AccountService.updateAccountBalance (6 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-ACCSVC-20	existing id	balance replaced	Pass	`AccountServiceTest #updateAccountBalance_existing`
TC-ACCSVC-21	unknown id	null, nothing saved	Pass	`AccountServiceTest #updateAccountBalance_missing`
TC-ACCSVC-22	Standard call	currency and owner are not changed	Pass	`AccountServiceTest #updateAccountBalance_keepsOtherFields`
TC-ACCSVC-32	negative balance	rejected without persistence	Pass	`AccountServiceTest #negativeAdjustmentRejected`
TC-ACCSVC-33	missing balance	rejected without persistence	Pass	`AccountServiceTest #nullAdjustmentRejected`
TC-ACCSVC-34	fractional cent	rejected without persistence	Pass	`AccountServiceTest #precisionAdjustmentRejected`

AccountService.getPaginatedAccounts (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-ACCSVC-23	page 0 size 2	page of DTOs	Pass	AccountServiceTest #getPaginatedAccounts_firstPage
TC-ACCSVC-24	page beyond data	empty page	Pass	AccountServiceTest #getPaginatedAccounts_emptyPage
TC-ACCSVC-25	Standard call	page/size forwarded as PageRequest	Pass	AccountServiceTest #getPaginatedAccounts_forwardsPageRequest

CustomerService

CustomerService.getCurrentCustomer (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-CUSSVC-01	existing user with profile	full DTO	Pass	CustomerServiceTest #getCurrentCustomer_found
TC-CUSSVC-02	unknown user	404 'User not found'	Pass	CustomerServiceTest #getCurrentCustomer_unknownUser
TC-CUSSVC-03	user without profile	404 'Customer not found'	Pass	CustomerServiceTest #getCurrentCustomer_noProfile

CustomerService.createCustomer (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-CUSSVC-04	valid DTO	saved with generated id	Pass	CustomerServiceTest #createCustomer_saves
TC-CUSSVC-05	Standard call	e-mail and phone are persisted	Pass	CustomerServiceTest #createCustomer_copiesContactDetails
TC-CUSSVC-06	profile created without login	username null	Pass	CustomerServiceTest #createCustomer_noLogin

CustomerService.getAllCustomers (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-CUSSVC-07	linked customer	contact details and username	Pass	CustomerServiceTest #listLinkedCustomer
TC-CUSSVC-08	no customers	empty list	Pass	CustomerServiceTest #listEmpty
TC-CUSSVC-09	profile without login	null username	Pass	CustomerServiceTest #listUnlinkedCustomer

CustomerService.getCustomerById (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-CUSSVC-10	linked profile	requested details	Pass	CustomerServiceTest #readLinkedCustomer
TC-CUSSVC-11	missing id	not found	Pass	CustomerServiceTest #readMissingCustomer
TC-CUSSVC-12	unlinked profile	null username	Pass	CustomerServiceTest #readUnlinkedCustomer

CustomerService.updateCustomer (4 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-CUSINT-02	Standard call	owner can edit self but cannot edit another profile	Pass	CustomerLifecycleIntegrationTest #ownerBoundaries
TC-CUSSVC-13	changed name and contacts	saved	Pass	CustomerServiceTest #updateContacts
TC-CUSSVC-14	body identity fields	stored identity unchanged	Pass	CustomerServiceTest #updatePreservesIdentity
TC-CUSSVC-15	missing id	not found and no save	Pass	CustomerServiceTest #updateMissingCustomer

CustomerService.deleteCustomer (6 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-CUSINT-01	eligible linked profile deleted	profile and login removed	Pass	`CustomerLifecycleIntegrationTest #adminDeletesLinkedProfile`
TC-CUSINT-03	customer with wallets	409 and profile retained	Pass	`CustomerLifecycleIntegrationTest #walletsProtectCustomer`
TC-CUSSVC-16	unlinked profile with no wallets	removed	Pass	`CustomerServiceTest #deleteUnlinkedCustomer`
TC-CUSSVC-17	linked profile without wallets	delete login with cascade	Pass	`CustomerServiceTest #deleteLinkedCustomer`
TC-CUSSVC-18	remaining wallet	409 and no deletion	Pass	`CustomerServiceTest #deleteCustomerWithWalletRejected`
TC-CUSSVC-19	unknown id	not found	Pass	`CustomerServiceTest #deleteMissingCustomer`

ExchangeRateService

ExchangeRateService.createRate (4 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-FXSVC-01	new USD->PHP 58.25	saved, scaled to 6 dp	Pass	`ExchangeRateServiceTest #createRate_saves`
TC-FXSVC-02	pair already exists	InvalidTransactionException	Pass	`ExchangeRateServiceTest #createRate_duplicate`
TC-FXSVC-03	same base and quote (PHP->PHP)	InvalidTransactionException	Pass	`ExchangeRateServiceTest #createRate_sameCurrency`
TC-FXSVC-04	unsupported currency JPY	InvalidTransactionException	Pass	`ExchangeRateServiceTest #createRate_unsupported`

ExchangeRateService.getAllRates (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-FXSVC-05	Standard call	rates returned sorted by base currency	Pass	`ExchangeRateServiceTest #getAllRates_sorted`
TC-FXSVC-06	empty table	empty list	Pass	`ExchangeRateServiceTest #getAllRates_empty`
TC-FXSVC-07	Standard call	DTO keeps id and rate value	Pass	`ExchangeRateServiceTest #getAllRates_mapsFields`

ExchangeRateService.getRateById (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-FXSVC-08	existing id	DTO	Pass	`ExchangeRateServiceTest #getRateById_found`
TC-FXSVC-09	unknown id	IllegalArgumentException 'Exchange rate not found'	Pass	`ExchangeRateServiceTest #getRateById_missing`
TC-FXSVC-10	Standard call	quote currency mapped correctly	Pass	`ExchangeRateServiceTest #getRateById_quoteMapped`

ExchangeRateService.updateRate (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-FXSVC-11	existing id	new value stored	Pass	`ExchangeRateServiceTest #updateRate_changesValue`
TC-FXSVC-12	unknown id	IllegalArgumentException, nothing saved	Pass	`ExchangeRateServiceTest #updateRate_missing`
TC-FXSVC-13	Standard call	currency pair in body is ignored (pair cannot change)	Pass	`ExchangeRateServiceTest #updateRate_keepsPair`

ExchangeRateService.deleteRate (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-FXSVC-14	existing id	deleted	Pass	`ExchangeRateServiceTest #deleteRate_existing`
TC-FXSVC-15	unknown id	IllegalArgumentException	Pass	`ExchangeRateServiceTest #deleteRate_missing`
TC-FXSVC-16	Standard call	only the requested id is deleted	Pass	`ExchangeRateServiceTest #deleteRate_onlyRequested`

ExchangeRateService.getRate (5 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-FXSVC-17	same currency PHP->PHP	1, no DB lookup	Pass	`ExchangeRateServiceTest #getRate_sameCurrency`
TC-FXSVC-18	direct USD->PHP 58 stored	58.000000	Pass	`ExchangeRateServiceTest #getRate_direct`
TC-FXSVC-19	only USD->PHP 50 stored, ask PHP->USD	inverse 0.020000	Pass	`ExchangeRateServiceTest #getRate_inverse`
TC-FXSVC-20	USD->EUR via PHP (USD->PHP 60, EUR->PHP 75)	0.800000	Pass	`ExchangeRateServiceTest #getRate_crossThroughPhp`
TC-FXSVC-21	no rate at all	IllegalArgumentException naming the pair	Pass	`ExchangeRateServiceTest #getRate_missing`

ExchangeRateService.convert (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-FXSVC-22	10 USD at 58	580.00 PHP	Pass	`ExchangeRateServiceTest #convert_usdToPhp`
TC-FXSVC-23	100 PHP to USD at 1/58	rounded HALF_EVEN to 1.72	Pass	`ExchangeRateServiceTest #convert_rounding`
TC-FXSVC-24	same currency	amount unchanged at 2 dp	Pass	`ExchangeRateServiceTest #convert_sameCurrency`

ExchangeRateService.getQuote (3 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-FXSVC-25	1000 PHP	EUR at 1/63 -> 15.87 EUR quote	Pass	`ExchangeRateServiceTest #getQuote_valid`
TC-FXSVC-26	amount 0	InvalidTransactionExceptio n	Pass	`ExchangeRateServiceTest #getQuote_zeroAmount`
TC-FXSVC-27	unsupported currency XYZ	InvalidTransactionExceptio n	Pass	`ExchangeRateServiceTest #getQuote_unsupported`

ForexRateSyncService

ForexRateSyncService.syncRates (6 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-FXSYNC-01	existing seeded USD->PHP 58	overwritten with 63.944000	Pass	`ForexRateSyncServiceTest #sync_updatesExistingRow`
TC-FXSYNC-02	empty table	4 new X->PHP rows inserted	Pass	`ForexRateSyncServiceTest #sync_insertsMissingRows`
TC-FXSYNC-03	PHP is the home currency	never fetched	Pass	`ForexRateSyncServiceTest #sync_skipsHomeCurrency`
TC-FXSYNC-04	Standard call	result carries provider name and ECB rate date	Pass	`ForexRateSyncServiceTest #sync_resultMetadata`
TC-FXSYNC-05	GBP fetch fails	others saved, GBP listed as failed	Pass	`ForexRateSyncServiceTest #sync_partialFailure`
TC-FXSYNC-06	provider unreachable	ForexProviderUnavailableE xception, nothing saved	Pass	`ForexRateSyncServiceTest #sync_allFail`

TransferService

TransferService.transferMoney (21 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-TRFINT-01	Alice sends 100 PHP to Bob	200, 900/600, one ledger row	**Pass**	`TransferIntegrationTest #happyPath`
TC-TRFINT-02	to missing account 999999	404 and debit ROLLED BACK	**Pass**	`TransferIntegrationTest #rollbackOnMissingDestination`
TC-TRFINT-03	Bob moves money out of Alice's wallet	403, nothing changes	**Pass**	`TransferIntegrationTest #bobCannotSpendAlicesMoney`
TC-TRFINT-04	1000 PHP into own USD forex wallet at 1/50	20.00 USD	**Pass**	`TransferIntegrationTest #forexIntoOwnDollarWallet`
TC-TRFINT-05	no token	401	**Pass**	`TransferIntegrationTest #noToken`
TC-TRFINT-08	zero amount	400 validation error	**Pass**	`TransferIntegrationTest #zeroAmountRejected`
TC-TRFINT-09	simultaneous debits on same snapshot	one commit, no lost debit	**Pass**	`TransferIntegrationTest #simultaneousDebits`
TC-TRFINT-10	simultaneous credits from different wallets	conflicting transfer entirely rolls back	**Pass**	`TransferIntegrationTest #simultaneousCredits`
TC-TRFINT-11	competing 700 transfers from 1000	only one succeeds, no overspend	**Pass**	`TransferIntegrationTest #simultaneousOverspendPrevented`
TC-TRFSVC-01	Alice sends 100.00 PHP to Bob	900/600 and one ledger row	**Pass**	`TransferServiceTest #transferMoney_happyPath`
TC-TRFSVC-02	unknown source	'Source account not found', nothing written	**Pass**	`TransferServiceTest #transferMoney_sourceMissing`
TC-TRFSVC-03	Bob sends from Alice's account	AccessDeniedException (403)	**Pass**	`TransferServiceTest #transferMoney_notOwner`
TC-TRFSVC-04	2000 from a 1000 balance	'Insufficient funds', nothing written	**Pass**	`TransferServiceTest #transferMoney_insufficientFunds`
TC-TRFSVC-05	destination 999 missing	'Destination account not found', no ledger row	**Pass**	`TransferServiceTest #transferMoney_destinationMissing`
TC-TRFSVC-06	1000 PHP into a USD forex wallet at 0.017241	17.24 USD credited	**Pass**	`TransferServiceTest #transferMoney_forexConversion`
TC-TRFSVC-07	fromId equals toId	InvalidTransactionExceptio n	**Pass**	`TransferServiceTest #transferMoney_sameAccount`
TC-TRFSVC-08	amount 0	InvalidTransactionExceptio n, DB untouched	**Pass**	`TransferServiceTest #transferMoney_zeroAmount`
TC-TRFSVC-09	amount 10.005 (3 decimals)	InvalidTransactionExceptio n	**Pass**	`TransferServiceTest #transferMoney_tooManyDecimals`
TC-TRFSVC-10	source account without an owner	AccessDeniedException	**Pass**	`TransferServiceTest #transferMoney_unownedSource`
TC-TRFSVC-11	no GBP->SGD rate	exception propagates, no credit, no ledger row	**Pass**	`TransferServiceTest #transferMoney_missingRate`
TC-TRFSVC-12	legacy wallets without currency treated as PHP	no conversion	**Pass**	`TransferServiceTest #transferMoney_legacyCurrencyDefau ltsToPhp`

TransferService.getTransferHistory (7 cases)

ID	Scenario (input)	Expected result	Result	Script
TC-TRFINT-06	Standard call	shows the successful transfer only, not the failed one	**Pass**	`TransferIntegrationTest #historyShowsOnlySuccessfulTransfers`
TC-TRFINT-07	Bob reads Alice's history	403; no token -> 401	**Pass**	`TransferIntegrationTest #historyIsPrivate`
TC-TRFSVC-13	owner	entries newest first, labelled SENT/RECEIVED	**Pass**	`TransferServiceTest #getTransferHistory_owner`
TC-TRFSVC-14	Bob reads Alice's history	AccessDeniedException, ledger not queried	**Pass**	`TransferServiceTest #getTransferHistory_notOwner`
TC-TRFSVC-15	unknown account	'Account not found'	**Pass**	`TransferServiceTest #getTransferHistory_missingAccount`
TC-TRFSVC-16	owner with no transfers	empty list	**Pass**	`TransferServiceTest #getTransferHistory_empty`
TC-TRFSVC-17	received forex entry	RECEIVED with both currencies, converted amount and rate	**Pass**	`TransferServiceTest #getTransferHistory_receivedForexEntry`

5. Getters and setters

`AccessorAndConstructorTest.setterThenGetterReturnsSameValue` is a parameterized test. For every setter/getter pair of every entity and DTO it sets three different sample values and asserts the getter returns each one (case `TC-POJO | Class.setX / getX | sample value #1..#3`). That gives each accessor three unique cases.

Sample values by type: Long 1, 42, Long.MAX_VALUE; String "PHP", "Juan dela Cruz", ""; BigDecimal 0.00, 100.50, 999999.99; LocalDateTime and LocalDate three distinct dates; List empty, one item, three items; int 0, 200, 404; Object a string, a number, a list; Customer and User three distinct instances.

85 accessor pairs, 255 executions, all passed. TC-POJO-CHK additionally fails the build if any entity or DTO has no accessor pair under test.

Class	Accessor pairs tested (setter / getter), 3 cases each	Result
Account	`setAccountType/getAccountType, setBalance/getBalance, setCurrency/getCurrency, setCustomer/getCustomer, setId/getId, setStatus/getStatus`	**Pass**
Customer	`setEmail/getEmail, setFullName/getFullName, setId/getId, setPhone/getPhone, setUser/getUser`	**Pass**
User	`setCustomer/getCustomer, setId/getId, setPassword/getPassword, setRole/getRole, setUsername/getUsername`	**Pass**
TransferRecord	`setAmount/getAmount, setConvertedAmount/getConvertedAmount, setDestinationAccountId/getDestinationAccountId, setDestinationCurrency/getDestinationCurrency, setExchangeRate/getExchangeRate, setId/getId, setSourceAccountId/getSourceAccountId, setSourceCurrency/getSourceCurrency, setTransferDate/getTransferDate`	**Pass**
ExchangeRate	`setBaseCurrency/getBaseCurrency, setId/getId, setQuoteCurrency/getQuoteCurrency, setRate/getRate`	**Pass**
AccountDTO	`setAccountType/getAccountType, setBalance/getBalance, setCurrency/getCurrency, setCustomerId/getCustomerId, setCustomerName/getCustomerName, setId/getId, setStatus/getStatus, setUsername/getUsername`	**Pass**
BalanceUpdateDTO	`setBalance/getBalance`	**Pass**
ApiResponse	`setData/getData, setMessage/getMessage, setStatus/getStatus, setTimestamp/getTimestamp`	**Pass**
CustomerDTO	`setEmail/getEmail, setFullName/getFullName, setId/getId, setPhone/getPhone, setUsername/getUsername`	**Pass**
LoginDTO	`setPassword/getPassword, setUsername/getUsername`	**Pass**
RegisterDTO	`setEmail/getEmail, setFullName/getFullName, setPassword/getPassword, setPhone/getPhone, setUsername/getUsername`	**Pass**
TransferRequestDTO	`setAmount/getAmount, setFromId/getFromId, setToId/getToId`	**Pass**
TransferRecordDTO	`setAmount/getAmount, setConvertedAmount/getConvertedAmount, setDestinationAccountId/getDestinationAccountId, setDestinationCurrency/getDestinationCurrency, setDirection/getDirection, setExchangeRate/getExchangeRate, setId/getId, setSourceAccountId/getSourceAccountId, setSourceCurrency/getSourceCurrency, setTransferDate/getTransferDate`	**Pass**
ExchangeRateDTO	`setBaseCurrency/getBaseCurrency, setId/getId, setQuoteCurrency/getQuoteCurrency, setRate/getRate, setUpdatedAt/getUpdatedAt`	**Pass**
ForexQuoteDTO	`setAmount/getAmount, setConvertedAmount/getConvertedAmount, setFromCurrency/getFromCurrency, setRate/getRate, setToCurrency/getToCurrency`	**Pass**
ForexSyncResultDTO	`setFailedCurrencies/getFailedCurrencies, setProvider/getProvider, setRateDate/getRateDate, setUpdatedRates/getUpdatedRates`	**Pass**
ErrorResponseDTO	`setErrorType/getErrorType, setMessage/getMessage, setStatus/getStatus, setTimestamp/getTimestamp`	**Pass**

6. Private helper methods

Private methods cannot be called from a test, so each is tested through the public method that calls it. Every helper below is executed by at least three of the named, passing cases from Section 4, each driving a different path through it.

Private method	What it does	Cases that execute it	Result
`FrankfurterClient.timeoutRequestFactory`	Builds the 5-second HTTP timeouts when Spring creates the client bean.	**TC-APP-01** GMoneyApplication.main: Standard call **TC-APP-02** GMoneyApplication.main: Standard call **TC-APP-03** GMoneyApplication.main: Standard call	**Pass**
`ForexRateSyncScheduler.syncQuietly`	Runs a sync and logs a provider outage instead of failing.	**TC-FXSCH-01** ForexRateSyncScheduler.syncOnStartup: application ready **TC-FXSCH-02** ForexRateSyncScheduler.syncOnStartup: provider down **TC-FXSCH-05** ForexRateSyncScheduler.syncOnSchedule: provider down	**Pass**
`AuthController.registerUser`	Shared registration logic: duplicate check, BCrypt hashing, linked User and Customer.	**TC-AUTHCTL-01** AuthController.register: new username **TC-AUTHCTL-02** AuthController.register: username taken **TC-AUTHCTL-04** AuthController.registerAdmin: admin creates admin	**Pass**
`TransferController.resolveUsername`	Takes the sender from the verified token, never the request body.	**TC-TRFCTL-01** TransferController.transferMoney: valid body + JWT user **TC-TRFCTL-07** TransferController.transferMoney: nobody logged in **TC-TRFCTL-08** TransferController.getTransferHistory: owner	**Pass**
`JwtUtil.getSigningKey`	Returns the HMAC key used to sign and verify tokens.	**TC-JWT-01** JwtUtil.generateToken: username alice **TC-JWT-04** JwtUtil.extractUsername: valid token **TC-JWT-05** JwtUtil.extractUsername: tampered signature	**Pass**
`JwtUtil.isTokenExpired`	Checks a token's expiry; runs only once the username matches.	**TC-JWT-07** JwtUtil.isTokenValid: token for alice checked against alice **TC-SEC-03** SecurityConfig.securityFilterChain: customer token on admin-only POST /forex/rates **TC-SEC-04** SecurityConfig.securityFilterChain: admin token on POST /forex/rates	**Pass**
`AccountService.convertToDTO`	Maps an Account entity to AccountDTO, including the owner's username used by @PostAuthorize.	**TC-ACCSVC-02** AccountService.getAccountById: existing id 1 **TC-ACCSVC-03** AccountService.getAccountById: owned USD wallet **TC-ACCSVC-10** AccountService.getAllAccounts: account without customer	**Pass**
`CustomerService.convertToDTO`	Maps a Customer to CustomerDTO, with the username when a login is linked.	**TC-CUSSVC-01** CustomerService.getCurrentCustomer: existing user with profile **TC-CUSSVC-07** CustomerService.getAllCustomers: linked customer **TC-CUSSVC-09** CustomerService.getAllCustomers: profile without login	**Pass**
`ExchangeRateService.findDirectOrInverse`	Looks up a stored rate in either direction (direct, else 1 / reverse).	**TC-FXSVC-18** ExchangeRateService.getRate: direct USD->PHP 58 stored **TC-FXSVC-19** ExchangeRateService.getRate: only USD->PHP 50 stored, ask PHP->USD **TC-FXSVC-20** ExchangeRateService.getRate: USD->EUR via PHP (USD->PHP 60, EUR->PHP 75)	**Pass**

Private method	What it does	Cases that execute it	Result
`ExchangeRateService.convertToDTO`	Maps an ExchangeRate entity to ExchangeRateDTO.	**TC-FXSVC-01** ExchangeRateService.createRate: new USD->PHP 58.25 **TC-FXSVC-07** ExchangeRateService.getAllRates: Standard call **TC-FXSVC-08** ExchangeRateService.getRateById: existing id	**Pass**
`ForexRateSyncService.upsert`	Inserts a new X->PHP row or overwrites the existing rate.	**TC-FXSYNC-01** ForexRateSyncService.syncRates: existing seeded USD->PHP 58 **TC-FXSYNC-02** ForexRateSyncService.syncRates: empty table **TC-FXSYNC-04** ForexRateSyncService.syncRates: Standard call	**Pass**
`TransferService.verifyOwnership`	Allows only the wallet's owner to send or read its history.	**TC-TRFSVC-01** TransferService.transferMoney: Alice sends 100.00 PHP to Bob **TC-TRFSVC-03** TransferService.transferMoney: Bob sends from Alice's account **TC-TRFSVC-14** TransferService.getTransferHistory: Bob reads Alice's history	**Pass**
`TransferService.validateAmount`	Rejects zero, negative and more-than-2-decimal amounts.	**TC-TRFSVC-01** TransferService.transferMoney: Alice sends 100.00 PHP to Bob **TC-TRFSVC-08** TransferService.transferMoney: amount 0 **TC-TRFSVC-09** TransferService.transferMoney: amount 10.005 (3 decimals)	**Pass**
`TransferService.currencyOf`	Reads a wallet's currency; blank legacy values count as PHP.	**TC-TRFSVC-01** TransferService.transferMoney: Alice sends 100.00 PHP to Bob **TC-TRFSVC-06** TransferService.transferMoney: 1000 PHP into a USD forex wallet at 0.017241 **TC-TRFSVC-12** TransferService.transferMoney: legacy wallets without currency treated as PHP	**Pass**
`TransferService.convertToDTO`	Maps a ledger row to TransferRecordDTO and labels it SENT or RECEIVED.	**TC-TRFSVC-13** TransferService.getTransferHistory: owner **TC-TRFSVC-17** TransferService.getTransferHistory: received forex entry **TC-TRFINT-06** TransferService.getTransferHistory (end to end): Standard call	**Pass**

7. Constructors

Constructors are not methods in Java, but they are listed for completeness. Dependency-injection constructors run whenever Spring or Mockito creates the object, so every test of that class exercises them; data-holder constructors are checked by the accessor and constructor tests.

Constructor	Arguments	How it is tested
`FrankfurterClient`	`RestClient.Builder builder, String baseUrl`	Called by Spring (TC-APP-01, TC-APP-02) or Mockito @InjectMocks before every case of its test class
`ForexRateSyncScheduler`	`ForexRateSyncService forexRateSyncService`	Called by Spring (TC-APP-01, TC-APP-02) or Mockito @InjectMocks before every case of its test class
`AccountController`	`AccountService accountService`	Called by Spring (TC-APP-01, TC-APP-02) or Mockito @InjectMocks before every case of its test class
`AuthController`	`AuthenticationManager authenticationManager, JwtUtil jwtUtil, UserRepository userRepository, PasswordEncoder passwordEncoder`	Called by Spring (TC-APP-01, TC-APP-02) or Mockito @InjectMocks before every case of its test class
`CustomerController`	`CustomerService customerService`	Called by Spring (TC-APP-01, TC-APP-02) or Mockito @InjectMocks before every case of its test class
`ForexController`	`ExchangeRateService exchangeRateService, ForexRateSyncService forexRateSyncService`	Called by Spring (TC-APP-01, TC-APP-02) or Mockito @InjectMocks before every case of its test class
`TransferController`	`TransferService transferService`	Called by Spring (TC-APP-01, TC-APP-02) or Mockito @InjectMocks before every case of its test class
`AccountDTO`	`no arguments`	Called by every TC-POJO accessor case for this class and by the service tests that build it
`ApiResponse`	`no arguments`	Called by every TC-POJO accessor case for this class and by the service tests that build it
`ApiResponse`	`int status, String message, T data`	2 named case(s): TC-CTOR-01
`ApiResponse`	`int status, String message`	2 named case(s): TC-CTOR-01
`CustomerDTO`	`no arguments`	Called by every TC-POJO accessor case for this class and by the service tests that build it
`CustomerDTO`	`Long id, String fullName, String email, String phone, String username`	1 named case(s): TC-CTOR-02
`CustomerDTO`	`Long id, String fullName`	1 named case(s): TC-CTOR-02
`ExchangeRateDTO`	`no arguments`	Called by every TC-POJO accessor case for this class and by the service tests that build it
`ForexQuoteDTO`	`no arguments`	Called by every TC-POJO accessor case for this class and by the service tests that build it
`ForexQuoteDTO`	`String fromCurrency, String toCurrency, BigDecimal amount, BigDecimal rate, BigDecimal convertedAmount`	1 named case(s): TC-CTOR-02
`ForexSyncResultDTO`	`no arguments`	Called by every TC-POJO accessor case for this class and by the service tests that build it
`ForexSyncResultDTO`	`String provider, LocalDate rateDate, List<ExchangeRateDTO> updatedRates, List<String> failedCurrencies`	Called by every TC-POJO accessor case for this class and by the service tests that build it
`LoginDTO`	`no arguments`	Called by every TC-POJO accessor case for this class and by the service tests that build it
`RegisterDTO`	`no arguments`	Called by every TC-POJO accessor case for this class and by the service tests that build it
`TransferRecordDTO`	`no arguments`	Called by every TC-POJO accessor case for this class and by the service tests that build it
`TransferRequestDTO`	`no arguments`	Called by every TC-POJO accessor case for this class and by the service tests that build it
`Account`	`no arguments`	Called by every TC-POJO accessor case for this class and by the service tests that build it

Constructor	Arguments	How it is tested
`Customer`	`no arguments`	Called by every TC-POJO accessor case for this class and by the service tests that build it
`ExchangeRate`	`no arguments`	Called by every TC-POJO accessor case for this class and by the service tests that build it
`ExchangeRate`	`String baseCurrency, String quoteCurrency, BigDecimal rate`	1 named case(s): TC-CTOR-02
`SupportedCurrency`	`String displayName`	Runs once per constant (PHP, USD, EUR, GBP, SGD) when the enum loads; display names verified by TC-CUR-07, TC-CUR-08, TC-CUR-09
`TransferRecord`	`no arguments`	Called by every TC-POJO accessor case for this class and by the service tests that build it
`User`	`no arguments`	Called by every TC-POJO accessor case for this class and by the service tests that build it
`DuplicateWalletException`	`String message`	Thrown and asserted in the service, controller and GlobalExceptionHandler cases
`ErrorResponseDTO`	`int status, String errorType, String message`	1 named case(s): TC-CTOR-03
`ForexProviderUnavailableException`	`String message`	Thrown and asserted in the service, controller and GlobalExceptionHandler cases
`InvalidTransactionException`	`String message`	Thrown and asserted in the service, controller and GlobalExceptionHandler cases
`AccountSecurity`	`UserRepository userRepository`	Called by Spring (TC-APP-01, TC-APP-02) or Mockito @InjectMocks before every case of its test class
`CustomAccessDeniedHandler`	`ObjectMapper objectMapper`	Called by Spring (TC-APP-01, TC-APP-02) or Mockito @InjectMocks before every case of its test class
`CustomAuthenticationEntryPoint`	`ObjectMapper objectMapper`	Called by Spring (TC-APP-01, TC-APP-02) or Mockito @InjectMocks before every case of its test class
`CustomUserDetailsService`	`UserRepository userRepository`	Called by Spring (TC-APP-01, TC-APP-02) or Mockito @InjectMocks before every case of its test class
`JwtAuthenticationFilter`	`JwtUtil jwtUtil, CustomUserDetailsService userDetailsService`	Called by Spring (TC-APP-01, TC-APP-02) or Mockito @InjectMocks before every case of its test class
`SecurityConfig`	`JwtAuthenticationFilter jwtAuthFilter, CustomUserDetailsService userDetailsService, CustomAuthenticationEntryPoint authEntryPoint, CustomAccessDeniedHandler accessDeniedHandler`	Called by Spring (TC-APP-01, TC-APP-02) or Mockito @InjectMocks before every case of its test class
`AccountService`	`AccountRepository accountRepository, CustomerRepository customerRepository, TransferRecordRepository transferRecordRepository`	Called by Spring (TC-APP-01, TC-APP-02) or Mockito @InjectMocks before every case of its test class
`CustomerService`	`CustomerRepository customerRepository, UserRepository userRepository, AccountRepository accountRepository`	Called by Spring (TC-APP-01, TC-APP-02) or Mockito @InjectMocks before every case of its test class
`ExchangeRateService`	`ExchangeRateRepository exchangeRateRepository`	Called by Spring (TC-APP-01, TC-APP-02) or Mockito @InjectMocks before every case of its test class
`ForexRateSyncService`	`FrankfurterClient frankfurterClient, ExchangeRateRepository exchangeRateRepository`	Called by Spring (TC-APP-01, TC-APP-02) or Mockito @InjectMocks before every case of its test class
`TransferService`	`AccountRepository accountRepository, TransferRecordRepository transferRecordRepository, ExchangeRateService exchangeRateService`	Called by Spring (TC-APP-01, TC-APP-02) or Mockito @InjectMocks before every case of its test class

8. Course regression suite

`Week5LabTest` is the course-provided Week 5 grading suite (security classes, JWT, login and registration). It is kept as a regression check: 52 checks, 52 passed.